Update a file through a Python algorithm

Project description

Within the organization, restricted content is controlled by granting certain IP addresses access to sensitive information. The file containing these granted IP addresses is within the `"allow_list.txt"` file. However, there are currently IP addresses within this list that must be removed.
The following Python algorithm was used to identify and remove IP addresses that do not have granted access to sensitive information and to fully automate and update the `"allow_list.txt"` file moving forward.

Open the file that contains the allow list

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"
```

```
# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:
```

In this algorithm, the `with` statement and `.open()` functions were used to open and read the `allow_list.txt` file.

The code `with open(import_file, "r") as file:`, the `open()` function has two parameters. The first parameter identifies the specific file to import. The second parameter indicates what should be done to the file – read. `File` will store output of the `.open()` function when working within the `with` statement.

Read the file contents

```
with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()
```

The `.read()` converts this file into string data. The `.read()` method was added to the `file` variable and was assigned to the string output to the created variable `ip_addresses`.

This code will read contents of the `"allow_list.txt"` file into string format.

Convert the string into a list

```
# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()
```

To find and remove individual IP addresses from the allow list, changing the string data into a list format allowed to easily identify the IP addresses became necessary. To perform this task, the `.split()` method was used to append the read contents of `ip_addresses` string into a list.
In this algorithm, the `.split()` function converts the stored data within the string data in `ip_addresses` and separates it into a list making use of whitespace.

Iterate through the remove list

```
# Build iterative statement
# Name loop variable `element`
# Loop through `remove_list`

for element in remove_list:
```

Next, it was important to this algorithm to iterate the IP addresses into the `remove_list`.

The `for` loop added here was to apply code statements to each element within a sequence. The `element` and `in` keyword indicates to iterate through the sequence `ip_addresses` and assign each value to the loop variable `element`.

Remove IP addresses that are on the remove list

```
for element in remove_list:

  # Create conditional statement to evaluate if `element` is in `ip_addresses`

    if element in ip_addresses:

      # use the `.remove()` method to remove
      # elements from `ip_addresses`

        ip_addresses.remove(element)
```

The algorithm at this point proved to identify IP addresses that were displayed in both the allowed list and removed lists. As this Python was able to show the reiterated IP addresses, the subsequent task was to remove the IP addresses that were produced in both lists.

Within the `for` loop, a conditional statement was created to ensure the loop variable `element` was found in the `ip_addresses` list. Then, the `.remove()` method was added to `ip_addresses` which allowed the loop variable `element` as the argument so that each IP address that was in the `remove_list` would be removed from `ip_addresses`.

Update the file with the revised list of IP addresses

```
# Convert `ip_addresses` back to a string so that it can be written into the text file

ip_addresses = "\n".join(ip_addresses)

  # Build `with` statement to rewrite the original file

  with open(import_file, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`

    file.write(ip_addresses)
```

To ensure the algorithm was fully viable, updating the allow list file was the final step to reflect the revised list of IP addresses being displayed.

The `.join()` method was used to convert all items back into a string, then, a new `with` statement was added to add write permissions with the `.write()` method to update the `"allow_list.txt"` file. The argument `"w"`, will call on the `.write()` function in the body of the `with` statement.

This allows that all restricted content will no longer be accessible to the IP addresses that were removed from the allow list.

Summary

This algorithm removes IP addresses identified in a `remove_list` variable from the `"allow_list.txt"` file of a list of approved IP addresses. The algorithm opened the file, converted it into a string to be read, and then converted the string into a list stored within a created variable, `ip_addresses`. The variable was iterated through the IP addresses in `remove_list`. If the IP addresses were listed in both the approved list and remove lists, then the `.remove()` method was used to remove the element from `ip_addresses`. Finally, the `.join()` method was used to convert the `ip_addresses` back into a string with write permissions to edit the `"allow_list.txt"` file with the revised list of IP addresses.